

LINKED LISTS AND EFFICIENCY

COMPUTER SCIENCE MENTORS 61A

March 31, 2026 – April 3, 2026

1 Linked Lists

For each of the following problems, assume linked lists are defined as follows:

```
class Link:
    empty = ()
    def __init__(self, first, rest=empty):
        assert rest is Link.empty or isinstance(rest, Link)
        self.first = first
        self.rest = rest

    def __repr__(self):
        if self.rest is not Link.empty:
            rest_repr = ', ' + repr(self.rest)
        else:
            rest_repr = ''
        return 'Link(' + repr(self.first) + rest_repr + ')'

    def __str__(self):
        string = '<'
        while self.rest is not Link.empty:
            string += str(self.first) + ' '
            self = self.rest
        return string + str(self.first) + '>'
```

1. What will Python output? Draw box-and-pointer diagrams along the way.

```
>>> a = Link(1, Link(2, Link(3)))
```

```
>>> a.first
```

```
>>> a.first = 5
```

```
>>> a.first
```

```
>>> a.rest.first
```

```
>>> a.rest.rest.rest.rest.first
```

```
>>> a.rest.rest.rest = a
```

```
>>> a.rest.rest.rest.rest.first
```

2. Write a function `skip`, which takes in a `Link` and skips every other element in the linked list.

(a) First, implement `skip` non-mutatively. That is, return a new linked list with every other element skipped, and do not modify the original linked list.

```
def skip(lst):
    """
    >>> a = Link(1, Link(2, Link(3, Link(4))))
    >>> a
    Link(1, Link(2, Link(3, Link(4))))
    >>> b = skip(a)
    >>> b
    Link(1, Link(3))
    >>> a
    Link(1, Link(2, Link(3, Link(4)))) # Unchanged
    """
    if _____:
        _____

    elif _____:
        _____

    _____
```

(b) Now, implement `skip` mutatively. That is, mutate the original list so that every other element is skipped. Do not call the `Link` constructor, and do not return anything.

```
def skip(lst):
    """
    >>> a = Link(1, Link(2, Link(3, Link(4))))
    >>> skip(a)
    >>> a
    Link(1, Link(3))
    """
```

3. Fill in the implementation of shuffle.

```
def shuffle(lnk):
    """Swaps each pair of items in a linked list destructively and returns
       the modified linked list.

    >>> shuffle(Link(1, Link(2, Link(3, Link(4)))))
    Link(2, Link(1, Link(4, Link(3))))
    >>> shuffle(Link('s', Link('c', Link(1, Link(6, Link('a'))))))
    Link('c', Link('s', Link(6, Link(1, Link('a')))))
    """

    if _____:
        return _____
    front = lnk.rest
    lnk.rest = _____
    _____
    return _____
```

4. Write a recursive function `insert_all` that takes as input two linked lists, `s` and `x`, and an index `index`. `insert_all` should return a new linked list with the contents of `x` inserted at index `index` of `s`.

```
def insert_all(s, x, index):
    """
    >>> insert = Link(3, Link(4))
    >>> original = Link(1, Link(2, Link(5)))
    >>> insert_all(original, insert, 2)
    Link(1, Link(2, Link(3, Link(4, Link(5)))))
    >>> start = Link(1)
    >>> insert_all(original, start, 0)
    Link(1, Link(1, Link(2, Link(5))))
    """

    if _____ and _____:
        _____

    if _____ and _____:
        _____
    _____
```

2 Efficiency and Orders of Growth

An order of growth characterizes the runtime **efficiency** of a program as its input becomes extremely large. Since we care about rate of growth, we ignore constant coefficients and exclusively consider the fastest growing term. For example, on very large inputs, $2n^2 + 3n - 20$ behaves the same as n^2 . Common runtimes, in increasing order of time, are: constant, logarithmic, linear, quadratic, and exponential.

Examples:

Constant time means that no matter the size of the input, the runtime of your program is consistent. In the function `f` below, no matter what you pass in for `n`, the runtime is the same.

```
def f(n):  
    return 1 + 2
```

A common example of a linear OOG involves a single for/while loop. In the example below, as `n` gets larger, the amount of time to run the function grows proportionally.

```
def f(n):  
    while n > 0:  
        print(n)  
        n -= 1
```

We can modify this while loop to get an example of logarithmic OOG. Suppose that, instead of subtracting 1 each time, we halve the size of `n`. For `n = 1000`, the program would take 10 iterations to terminate (since $2^{10} = 1024$). The runtime is proportional to $\log(n)$.

```
def f(n):  
    while n > 0:  
        print(n)  
        n //= 2
```

An example of a quadratic runtime involves nested for loops. For every one of the `n` iterations of the outer loop, there is `n` work done in the inner loop. This means that the runtime is proportional to n^2 .

```
def f(n):  
    for i in range(n):  
        for j in range(n):  
            print(i*j)
```

1. Give a tight asymptotic runtime bound for the following functions in $\Theta(\cdot)$ notation, or “Infinite” if the program does not terminate.

(a) **def** one(n):
 while n > 0:
 n = n // 2

(b) **def** two(n):
 for i **in** range(n):
 for j **in** range(i):
 print(str(i), str(j))

(c) **def** three(n):
 i = 1
 while i <= n:
 for j **in** range(i):
 print(j)
 i *= 2

2. Say we want to repeatedly insert some numbers to the end of a linked list.

```
def append_many(s, items):  
    for item in items:  
        append(s, item)
```

- (a) Assuming s is initially length 1. How long will it take to complete the first insertion? The second? The n th?
- (b) Give the total runtime in $\Theta(\cdot)$ notation if s is initially empty and $items$ contains n items.

3. Consider the following linked list function.

```
def prepend(s, item):  
    return link(item, s)
```

- (a) What does this function do?
- (b) Assume s is initially length n , how long does it take to prepend once? prepend twice? prepend n times?